INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Excelencia en Educación Superior

3.1 HoldOut

python™

Erick Mejia

In [13]:
```
import numpy as np
from sklearn import datasets
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

In [14]:
```
# Carga de datos
iris = datasets.load_iris()
print(iris.target_names)
```

```
['setosa' 'versicolor' 'virginica']
```

In [15]:
```
# Mostrar características de la tabla de datos.
print("Tabla de datos: %d instancias y %d atributos" % (iris.data.shape[0], iris.da
print("Valores de la clase:", set(iris.target))

# Cuantificamos el número de instancias que contiene el dataset por clase
valores, ocurrencias = np.unique(iris.target, return_counts=True)
print(valores, ocurrencias)
```

```
Tabla de datos: 150 instancias y 4 atributos
Valores de la clase: {np.int64(0), np.int64(1), np.int64(2)}
[0 1 2] [50 50 50]
```

In [16]:
```
# Test: hold-out split 80-20%. # Partición externa
X_training, X_test, y_training, y_test = train_test_split(iris.data, iris.target, t
```

```
valores_test, ocur_test = np.unique(y_test, return_counts=True)
print('Test: ', 'clases:', valores_test, ' ocurrencias: ', ocur_test)
```

```
Test:  clases: [0 1 2]  ocurrencias:  [10  9 11]
```

In [17]:
```
# Estandarizar las características de entrenamiento y de test
standardizer = StandardScaler()
X_training = standardizer.fit_transform(X_training)
X_test = standardizer.transform(X_test)
```

In [18]:
```
# Validación: hold-out split 80-20%. # Partición interna
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size
valores_train, ocur_train = np.unique(y_train, return_counts=True)
print('Entrenamiento: ', ' clases:', valores_train, '  ocurrencias:', ocur_train)

valores_val, ocur_val = np.unique(y_val, return_counts=True)
print('Validation:    ', ' clases:', valores_val, '  ocurrencias:', ocur_val)
```

```
Entrenamiento:   clases: [0 1 2]   ocurrencias: [32 30 34]
Validation:      clases: [0 1 2]   ocurrencias: [ 8 11  5]
```

In [19]:
```
# Construcción del objeto que contiene el algoritmo de aprendizaje.
clf = DummyClassifier(strategy='prior', random_state=42)
```

In [20]:
```
# Entrenamiento del algoritmo de aprendizaje.
clf = clf.fit(X_train, y_train)
```

In [21]:
```
# Evaluación del algoritmo de aprendizaje con el método "score" que devuelve direct
val_accuracy = clf.score(X_val, y_val)
print("Exactitud en validación: ", val_accuracy)

test_accuracy = clf.score(X_test, y_test)
print("Exactitud en test: ", test_accuracy)
```

```
Exactitud en validación:  0.20833333333333334
Exactitud en test:  0.36666666666666664
```

In [22]:
```
# Obtenemos las predicciones sobre conjunto de validación y de test
y_pred_val = clf.predict(X_val)
print('Predicciones de validación ', y_pred_val)
print('Etiquetas reales validación', y_val)

y_pred_test = clf.predict(X_test)
print('\nPredicciones de test ', y_pred_val)
print('Etiquetas reales test', y_val)
```

```
Predicciones de validación  [2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
Etiquetas reales validación [1 1 0 0 0 2 1 2 2 2 1 1 1 1 1 0 2 0 1 0 1 1 0 0]

Predicciones de test  [2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
Etiquetas reales test [1 1 0 0 0 2 1 2 2 2 1 1 1 1 1 0 2 0 1 0 1 1 0 0]
```

In [23]:
```
# Aplicamos un ejemplo con un clasificador más complejo que el "dummyclassifier"
from sklearn.svm import SVC
svc = SVC(C=0.5) # Definimos algoritmo
svc.fit(X_train, y_train) # Entrenamos modelo
```

```python
val_accuracy = svc.score(X_val, y_val) # Evaluamos modelo en validación
print('Exactitud en validación ', np.round(val_accuracy*100, 4), '%')

test_accuracy = svc.score(X_test, y_test) # Evaluamos modelo en test
print('Exactitud en test ', np.round(test_accuracy*100, 4), '%')
```

```
Exactitud en validación  91.6667 %
Exactitud en test  96.6667 %
```

In [26]:

```python
# Guardar modelo
import pickle
with open('models/model.pickle', 'wb') as fw:
    pickle.dump(svc, fw)

# Cargar modelo
with open('models/model.pickle', 'rb') as fr:
    pickle.load(fr)
```

GitHub

https://github.com/Erick-305/machine-learning.git